Listing 3.16 Updating the `<Chat />` component that handles file uploads

```
const [imageUrl, setImageUrl] =
          React.useState(null);
const handleUploadFile = async (file, base64) => {
  setImageUrl(base64);
};
const handleOnSubmit = (e) => {
  e.preventDefault();
  const data = { message: e.target.value };
  if (imageUrl) {.
    data.imageUrl = imageUrl;
    setImageUrl(null);
  }
  handleSubmit(e, { data });
};
```

Initializes the state variable imageUrl to null using the useState hook to manage the URL of the uploaded image

Constructs an object data to hold the form data, including the text message extracted from the event

Checks if an image URL is available. If imageUrl is not null (i.e., an image has been uploaded), it proceeds to add the image URL to the data object.

This function handles the form submission, sending the message and the image URL to the appropriate handler.

On the backend, the `processRequestMessages` function extends the chat message format so the last user prompt can carry both text and image data. This allows the Gemini model to receive richer, multimodal input rather than plain text. On the frontend, the `<Chat />` component is updated to include an optional image upload, preview, and submission flow. When a user sends a message with an image, the form handler bundles both into the request body, ensuring they are processed together.

The component that handles all the logic for uploading an image into a string is the `<FileUploader />` component:

```
<FileUploader
     onFileUpload={handleUploadFile}
     maxFileSize = {10 * 1024 * 1024}
/>
```

This component utilizes the DOM API to read the selected file and convert it into a Base64 string. Since this functionality is encapsulated within the component, we won't explore more of its implementation details.

The reasoning behind those changes is to make conversations more powerful and natural: users aren't limited to words; they can upload an image and ask the model to interpret, describe, or analyze it. This capability turns Astra AI into a true multimodal assistant, widening the scope of use cases and making the chat experience more interactive and engaging.

Now let's test these changes. Reload the application and utilize the file upload functionality to upload an image. Then prompt the AI agent to describe what it sees. An example interaction is illustrated in figure 3.8, where the AI agent's response proves to be accurate. This integration enhances the user experience by allowing seamless image uploads and seamless responses from the AI agent.